

Web Application Framework (WAF) - Fall 2026

Task - 1 (18th September 2026)

Part-1: Domain Name Classification

Use [Mockaroo - Random Data Generator and API Mocking Tool | JSON / CSV / SQL / Excel](#) to generate 1000 instances of JSON data for first and last name, gender, email, IP addresses. Your task is to create four files, named "com.txt", "gov.txt", "edu.txt", and "uk.txt". Find instances from the email address field of the generated data having .com, .edu, .org, and .uk. Append the complete instance of the record to the relevant files created.

Part-2: Asynchronous Function Execution

Your program should define and call six functions named `register`, `sendWelcomeMessage`, `login`, `fetchProfile`, `updateStatus`, and `logout`. Use the asynchronous `setTimeout` function to introduce delays in each. Each function should output a console message indicating task completion. Call these functions in the following order with the specified delays, and the last line of code should be. `console.log("All operations finished!")`

- `register` = delay of 2500 milliseconds (2.5 seconds)
- `sendWelcomeMessage` = delay of 3000 milliseconds (3 seconds)
- `login` = delay of 2000 milliseconds (2 seconds)
- `fetchProfile` = delay of 4000 milliseconds (4 seconds)
- `updateStatus` = delay of 1500 milliseconds (1.5 seconds)
- `logout` = delay of 3500 milliseconds (3.5 seconds)

- Call these functions in the above order with the given delays and observe the execution order. Note the sequence in which messages appear.
- Use callbacks to ensure the functions are executed in the specified order. Call them again with the same delays and verify the order.
- Use Promises to achieve the same ordered execution as in part ii. Call the functions with the delays and confirm the sequence.
- Use `async/await` to accomplish the same as part ii or iii. Call the functions with the delays and verify the execution order.

Part-3: IP Address Classes and JSON Data

Use [Mockaroo - Random Data Generator and API Mocking Tool | JSON / CSV / SQL / Excel](#) to generate 1000 instances of JSON data for first and last name, gender, email, and IP addresses. Read the generated JSON file. Identify the objects of class A, B, C, D, and E IP addresses based on the subnet mask and store the data in five different files (`IP_Class_A.txt`, `IP_Class_B.txt`, `IP_Class_C.txt`, `IP_Class_D.txt`, `IP_Class_E.txt`), one each for a class of IP

addresses.

Notes

- i. Refer to NodeJs official documentation as needed.
- ii. Place each part code in separate files and place it in a folder with a name such as yourRollNo_yourName_Task1_F26. Compress the folder.
- iii. Submit the above-compressed folder on fs to the folder WAF-F26-T1